

TEXTBRIEF AI

Техническа документация на учебен модел за извличащо съкращаване на текст

Какво обхваща: подготовка на данни, речник и token IDs, embeddings, masked mean pooling, двукласова класификация, обучение с PyTorch, checkpoint, inference, избор на изречения и експериментални резултати.

Статус на проекта. Завършен минимален AI прототип. Резултатите по-долу демонстрират работещ процес, а не доказано качество на професионален summarizer.

Версия: 1.0 | 17 септември 2026 г.

Език на документацията: български | Примери и коментари в кода: английски

1. Какво е TextBrief и каква е целта му?

TextBrief е учебно приложение, което съкращава текст чрез **избор на изречения от оригинала** (extractive summarization). То не генерира нови формулировки: всяко изречение получава оценка за важност, след което се запазват зададен брой изречения с най-висок резултат.

Основната цел е да се премине през пълен, разбираем AI цикъл - от подготвянето на данните до използването на обучен модел. Проектът е демонстрационен, а не готов за произволни документи.

Път на данните

ВХОДЕН ТЕКСТ
Разделяне на изречения
Tokenization → vocabulary → IDs → padding
Embedding → masked mean pooling → Linear
Logit → sigmoid → вероятност за клас 1
Избор на top_k → оригинален ред → РЕЗЮМЕ

Какво предсказва моделът?

За **всяко отделно изречение** моделът връща един logit - сурова числова оценка за клас **1** = **важно**. Sigmoid преобразува logit-a в число между 0 и 1. При показване на двукласово предсказание ползваме праг 0.5: стойност поне 0.5 → 1; под 0.5 → 0.

Съществено ограничение. Моделът не е context-aware: не вижда останалите изречения в документа, когато оценява текущото. „Важно“ в учебните данни означава приблизително „съществено за избраната AI тема“, а не универсална важност във всеки контекст.

Ориентир за структурата

```
AI/
  data/raw/training.csv
  src/data/preprocessing.py
  src/data/dataset.py
  src/model/summarizer.py
  src/training/train.py
  src/inference/predict.py
  src/inference/summarize.py
  tests/test_model.py
  artifacts/textbrief_model.pt
```

Пътищата в примерите по-долу са спрямо директорията **AI/**. Файлът с checkpoint се появява след успешно обучение.

2. Данните: изречения и labels

Обучаващият CSV има две колони: **sentence** и **label**. Една стойност 1 обозначава важно изречение, а 0 - неважно според нашите учебни критерии. `pandas.read_csv()` зарежда редовете, а `TextSummaryDataset` предоставя примерите на PyTorch.

```
sentence,label
PyTorch is a machine learning framework.,1
It was originally developed by Meta.,0
PyTorch uses tensors to represent numerical data.,1
The tutorial uses a blue background.,0
```

Това са илюстративни редове от използваните в проекта учебни примери, не пълно копие на локалния CSV. По-късно добавихме още примери за tensors, optimizers, embeddings и странични подробности.

Защо повече данни?

Първоначалните четири примера бяха достатъчни за проверка дали кодът се изпълнява. Разширеният CSV добавя повече думи и примери, но сам по себе си не гарантира обобщаване. Нужни са разнообразни и последователно означени данни.

3. Preprocessing: от текст към числа

3.1. tokenize(text)

Преобразува текста в малки букви и го разделя по интервали. Така еднакви думи с различни главни букви стават еднакви token-и. Това е нарочно проста първа версия.

```
"PyTorch uses tensors." → ["pytorch", "uses", "tensors."]
```

Важно. Пунктуацията остава част от token-а: "learning" и "learning." са различни token-и. По-добра токенизация е възможно бъдещо подобрене.

3.2. build_vocabulary(sentences)

Изгражда речник **token → integer ID**. Два специални token-а имат фиксирани ID-та: `<PAD> = 0` и `<UNK> = 1`. За всяка нова дума се добавя следващото свободно ID. Номерата са **идентификатори**, а не числови оценки на значението на думите.

```
{"<PAD>": 0, "<UNK>": 1, "pytorch": 2, "uses": 3, ...}
```

3.3. encode_sentence(sentence, vocabulary)

Превръща token-ите в списък от ID-та. Познатите получават своите ID-та от речника, а непознатите - ID-то на `` (1). При inference не разширяваме речника, защото обучените embedding-и са свързани с конкретните ID-та.

3.4. pad_sequence(token_ids, max_length, pad_id=0)

Допълва по-кратките последователности с нули до обща дължина. Така изреченията могат да се групират в tensor с еднакви размери. `` означава „тук няма истински token“, а `` означава „има истинска дума, но не я познаваме“.

```
[2, 3, 8] --pad to length 6--> [2, 3, 8, 0, 0, 0]
```

В текущата версия `max\_length = 10`. Повече обучаващи изречения **не увеличават автоматично** този лимит. `predict\_probability()` отказва изречения с повече от 10 token-а, а не ги съкращава тайно.

3.5. Dataset и DataLoader

`TextSummaryDataset.\_\_getitem\_\_()` връща двойка `(sentence\_tensor, label\_tensor)`. Token ID-тата са `torch.long`, защото `nn.Embedding` очаква целочислени индекси; labels са `torch.float32`, защото `BCEWithLogitsLoss` работи с плаваща запетая.

```
sentence_tensor: torch.Size([10])
label_tensor: scalar float, e.g. tensor(1.)

DataLoader(batch_size=2) -> sentences: [2, 10], labels: [2]
```

`DataLoader` подава batch-ове, а `shuffle=True` разбърква реда на примерите по време на обучението. Batch размер 2 означава **две изречения в един batch**, не „два batch-а“.

4. Tensor shapes: кратък справочник

В PyTorch измеренията се номерират от 0, както индексите в Python. При shape `[2, 10, 16]`: `dim=0` е измерението за 2 изречения; `dim=1` е за 10 token позиции; `dim=2` е за 16 embedding стойности.

Операция / форма	Какво означава
label_batch.size(0)	Размерът на първото измерение - броят изречения в batch-а.
sum(dim=1)	Сумиране по второто измерение; тук по token позициите.
unsqueeze(-1)	Добавя последно измерение: [B, L] → [B, L, 1].
unsqueeze(0)	Добавя първо, batch измерение: [L] → [1, L].
squeeze(-1)	Маха последно измерение с размер 1: [B, 1] → [B].

Не бъркай индекс и размер. `dim=0` е номерът на измерението; при `shape [2, 10]` неговият размер е 2. Shapes описват размери, не списъци от индекси.

5. Архитектура на модела

5.1. Embedding: token ID → вектор

`nn.Embedding(vocab_size, embedding_dim, padding_idx=0)` пази обучаемата таблица с форма `[vocab_size, embedding_dim]`. ID-то на token-а избира един ред от нея. При `embedding_dim=16` всеки token има вектор от 16 обучаеми числа. ID-тата не означават „по-голямо значение“: те са само адреси в таблицата.

```
Input token IDs:      [batch_size, 10]
Embedding output:     [batch_size, 10, 16]
Embedding weight table: [vocab_size, 16]
```

`padding_idx=0` задава специалния PAD ред: при стандартната инициализация той е нулев и не се обновява чрез градиентите от Embedding. Реалните token embedding-и се настройват по време на обучението.

5.2. Masked mean pooling: един вектор за изречение

След Embedding имаме по един вектор за **всеки token**. За да класифицираме изречението, ги обединяваме в **един вектор с 16 стойности**. Използваме средно аритметично само на реалните token-и; PAD не участва в броя.

```
embeddings = self.embedding(x)           # [B, L, D]
mask = (x != self.padding_idx).unsqueeze(-1) # [B, L, 1]
masked_embeddings = embeddings * mask
summed_embeddings = masked_embeddings.sum(dim=1) # [B, D]
tokens_count = mask.sum(dim=1)              # [B, 1]
sentence_vectors = summed_embeddings / tokens_count.clamp(min=1)
```

Маската е `True` за истински token и `False` за PAD. `unsqueeze(-1)` позволява маската да се умножи по 3D embeddings чрез **broadcasting**. `sum(dim=1)` сумира съответните 16 компонента през token позициите; `mask.sum(dim=1)` брои реалните token-и.

```
Real tokens: [2, 4] and [4, 8]; PAD: [0, 0]
Sum:         [6, 12]
Token count: 2
Mean:        [3, 6]
```

Използваме **средно, а не сума**, защото при сумиране по-дългите изречения могат да получават по-големи стойности само заради броя думи. Средното намалява този ефект, макар да не премахва напълно влиянието на дължината. `clamp(min=1)` предпазва от деление на нула при празен вход.

Цената на простотата. Mean pooling губи словореда: различни подредби на същите token-и имат еднакъв усреднен вектор. Моделът не използва внимание (attention) или контекст от съседни изречения.

5.3. Linear класификатор

``nn.Linear(embedding_dim, 1)`` преобразува 16-те стойности на изречението в **един logit**. За едно изречение операцията има вида $z = w_1x_1 + \dots + w_{16}x_{16} + b$: 16 обучаеми тегла, едно bias число и 16 входни стойности. ``squeeze(-1)`` премахва излишното последно измерение.

```
[B, 16] --Linear(16, 1)--> [B, 1] --squeeze(-1)--> [B]
```

При ``batch_size=1`` ``squeeze(-1)`` запазва batch измерението (``[1, 1] → [1]``); обикновеното ``squeeze()`` би могло да премахне и него.

6. Logits, sigmoid и предсказания

Logit е суровата оценка на класификатора, не вероятност и не „важност от 1 до 10“. Може да е всяко реално число. Sigmoid го превръща в стойност между 0 и 1, която използваме като предсказана вероятност за клас 1.

Logit	Sigmoid(logit) - приблизително
-2	0.119 - по-ниска оценка за клас 1
0	0.500 - гранична оценка
2	0.881 - по-висока оценка за клас 1

```
probability = torch.sigmoid(logits)
prediction = (probability >= 0.5).int()
```

При ``probability = 0.500`` визуалното закръгляне до три знака може да скрие реалната стойност. За отстраняване на съмнения може временно да отпечатаме шест знака. Прагът 0.5 е наше правило за преобразуване на вероятността в клас, не абсолютен критерий за важност.

7. Обучение: loss, gradients, optimizer

7.1. Criterion и BCEWithLogitsLoss

`criterion` е просто име на променлива, която съдържа функцията за загуба. В TextBrief използваме `nn.BCEWithLogitsLoss()`, подходяща за бинарни labels 0/1. Тя сравнява **суровите logits** с истинските labels, включва sigmoid вътрешно по числено стабилен начин и по подразбиране връща **среден loss за текущия batch**. Не прилагаме sigmoid преди тази loss функция.

```
criterion = nn.BCEWithLogitsLoss()
logits = model(sentence_batch)
loss = criterion(logits, label_batch)
```

Loss е числена мярка за несъответствието между предсказанията и labels, **не е процент точност**. При сравнение върху същите данни по-нисък loss означава по-добро напасване по този критерий, но не гарантира качество върху непознати текстове.

7.2. Една training step

```
optimizer.zero_grad() # Clear gradients left from previous steps.
logits = model(sentence_batch)
loss = criterion(logits, label_batch)
loss.backward() # Compute gradients for trainable parameters.
optimizer.step() # Update weights, biases, and embeddings.
```

PyTorch натрупва градиентите по подразбиране; затова `zero\_grad()` ги изчиства за новата стъпка. `backward()` изчислява как loss се изменя спрямо параметрите, но **не ги променя**. `optimizer.step()` действително обновява научаемите параметри. За този първи проект е достатъчно да разбираме ролите им; подробната математика на производните и backpropagation остава за курсовете.

7.3. Adam и learning rate

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

`model.parameters()` включва embedding векторите, теглата и bias на Linear слоя. Adam използва изчислените градиенти, за да избере обновяванията; `lr=0.01` задава мащаба на стъпките. Обновяването не гарантира, че loss ще намалее при всяка отделна стъпка.

8. Batch, epoch и среден loss

Batch е група примери, обработена заедно. **Training step** е едно преминаване на batch през forward → loss → backward → optimizer. **Epoch** е едно пълно преминаване през всички тренировъчни примери.

Първоначалният учебен тест използваше 4 изречения с `batch\_size=2`: 2 batch-a в един epoch и 60 training steps за 30 epochs. След разширяването на CSV броят steps на epoch

зависи от новия брой редове и размера на batch-a.

Как изчисляваме средния loss за цял epoch?

```
total_loss = 0.0
total_samples = 0

for sentence_batch, label_batch in dataloader:
    # ... forward, loss, backward, optimizer.step() ...
    total_loss += loss.item() * label_batch.size(0)
    total_samples += label_batch.size(0)

average_loss = total_loss / total_samples
```

`loss.item()` взема Python число от tensor с един елемент. `label\_batch.size(0)` е броят изречения в текущия batch, защото `dim=0` е batch измерението. Умножаваме средния batch loss по броя примери, сумираме и делим на общия брой изречения - така изчислението е коректно и когато последният batch е по-малък.

```
Batch A: 2 sentences, mean loss 0.4 -> 0.4 * 2 = 0.8
Batch B: 2 sentences, mean loss 0.6 -> 0.6 * 2 = 1.2
Epoch mean loss = (0.8 + 1.2) / 4 = 0.5
```

Какво видяхме при обучение?

В първоначален тест с четири изречения loss намаля от приблизително **0.7056** на epoch 1 до **0.0439** на epoch 30. След отделянето на `train.py` друг пуск показва намаляване от **0.7061** до **0.0646**. Това са конкретни демонстрационни изпълнения; стойностите се различават заради случайна инициализация и разбъркване на данните.

Какво доказва това? Моделът може да намали загубата върху тренировъчните примери. Това не е оценка на generalization и не бива да се представя като точност на непознати текстове.

9. Checkpoint: запазване и възстановяване

В края на `src/training/train.py` записваме checkpoint: `artifacts/textbrief\_model.pt`. Това е файлът, който **нашият код създава** чрез `torch.save()`; не идва от външен готов модел.

```
torch.save({
    "model_state_dict": model.state_dict(),
    "vocabulary": vocabulary,
    "embedding_dim": 16,
    "max_length": 10,
    "padding_idx": 0,
}, "artifacts/textbrief_model.pt")
```

`state\_dict()` съдържа научените параметри. Vocabulary се пази с тях, защото при inference думите трябва да имат същите ID-та като при обучението. Не запазваме градиентите за inference: те описват конкретна тренировъчна стъпка, а не наученото. За точно

продължение на обучение обикновено се пази и optimizer state.

10. Inference: използване без ново обучение

Training променя параметрите. **Inference** използва вече обучените параметри за предсказване. В проекта `src/inference/predict.py` има две функции: `load\_predictor()` възстановява модела веднъж, а `predict\_probability()` обработва отделно изречение.

10.1. Зареждане

```
checkpoint = torch.load(
    "artifacts/textbrief_model.pt",
    map_location="cpu",
    weights_only=True
)
model = TextSummarizer(
    vocab_size=len(checkpoint["vocabulary"]),
    embedding_dim=checkpoint["embedding_dim"],
    padding_idx=checkpoint["padding_idx"]
)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()
```

`torch.load()` прочита запазения речник, настройки и параметри. `load\_state\_dict()` поставя научените параметри в новосъздадения модел. `map\_location="cpu"` зарежда данните за работа на CPU. `weights\_only=True` ограничава типовете обекти при зареждане; checkpoint-и се зареждат само от доверени източници.

10.2. Подготовка на едно изречение

Отново използваме същия tokenizer, речник и padding като при обучение. `unsqueeze(0)` добавя batch измерение: едно изречение с форма `[10]` става `[1, 10]`. Това позволява `sum(dim=1)` да продължи да означава сумиране по token-ите.

```
sentence_tensor = torch.tensor(token_ids, dtype=torch.long).unsqueeze(0)
# [max_length] -> [1, max_length]

with torch.no_grad():
    logit = model(sentence_tensor)
    probability = torch.sigmoid(logit).item()
```

`model.eval()` превключва слоевете в режим за оценяване, а `torch.no\_grad()` спира проследяването за градиенти. При текущите Embedding, mean pooling и Linear `eval()` практически не променя изхода, но е коректният режим за inference и ще бъде важен при слоеве като Dropout.

10.3. Проверени примери

Въведено изречение	Наблюдаван резултат при конкретен пуск
PyTorch is a machine learning framework.	Probability 0.980 → клас 1.
It was originally developed by Meta.	Probability 0.091 → клас 0.
I went to the mall.	Probability 0.500 (закръглено) → клас 1.

Първите два примера бяха съпоставени с използваните тренировъчни labels. Изречението за мола показва защо границата 0.5 и малкият речник не са надеждна „семантична преценка“. Това са наблюдения от ранния checkpoint; след ново обучение числата могат да се променят.

11. Съкращаване на цял текст

`src/inference/summarize.py` разделя входния текст на изречения с опростен regex, извиква `predict\_probability()` за всяко изречение и предава получените вероятности на `select\_summary()`.

```
sentences = [
    part.strip()
    for part in re.split(r"(?<=[.!?])\s+", text.strip())
    if part.strip()
]
probabilities = [
    predict_probability(s, model, checkpoint) for s in sentences
]
return select_summary(sentences, probabilities, top_k=2)
```

Регулярният израз разделя след точка, удивителна или въпросителна, ако има следващ whitespace. Това не е пълноценен sentence tokenizer и може да се затрудни със съкращения или необичайна пунктуация.

12. Как точно избираме резюмето?

```
def select_summary(sentences, probabilities, top_k=2):
    ranked_indices = sorted(
        range(len(sentences)),
        key=lambda i: probabilities[i],
        reverse=True
    )
    selected_indices = ranked_indices[:top_k]
    return " ".join(sentences[i] for i in selected_indices)
```

Първото сортиране подрежда **индексите** по важност, а не самите изречения. `ranked\_indices[:top\_k]` взема индексите на най-високо оценените. Второто сортиране ги подрежда по първоначалния им индекс, за да запазим оригиналния ред. ` " ".join(...)` съединява избраните изречения с интервали.

```
Sentences:      [A, B, C, D]
Probabilities:  [0.3, 0.9, 0.2, 0.8]
Ranked IDs:    [1, 3, 0, 2]
Selected IDs:   [1, 3]
Summary:       B D
```

`top\_k=2` означава „най-много две изречения“. Ако входът съдържа само две, функцията ще върне и двете - това само по себе си **не е съкращаване**. С три или повече изречения вече можем да проверим реален избор.

13. Наблюдавани тестове и как ги тълкуваме

13.1. Познати обучаващи изречения

След разширяване и повторно обучение подадохме текст от пет учебни изречения. Моделът избра:

```
PyTorch uses tensors to represent numerical data.
An optimizer updates model parameters during training.
```

И двете бяха означени с 1 в нашия учебен CSV. Това потвърждава, че работят възстановяването на модела и изборът на изречения. Понеже изреченията са от training data, тестът **не доказва работа с непознат текст**.

13.2. Нови формулировки

При следващ тест подадохме пет изречения, които не се срещат дословно в разширения учебен CSV. Резюмето съдържаше:

```
Training uses examples to update model parameters.
Neural networks predict labels from numerical data.
```

Това показва, че моделът не разчита единствено на буквално съвпадение с цели тренировъчни изречения. Но много token-и и теми остават познати от обучението, а част от непознатите думи стават ``. Един успешен пример не доказва разбиране на смисъл или надеждна генерализация.

13.3. Какво все още не сме измерили?

Не сме провели систематично измерване на accuracy / precision / recall върху предварително отделен тестов набор. Не трябва да измисляме такива стойности или да наричаме понижения тренировъчен loss „тестова точност“.

14. Как да направим честна проверка с нови данни

Следващо малко подобрение е да разделим означените данни **преди обучението**. Част от документите/изреченията служат за обучение, друга - за проверка. Vocabulary трябва да се строи **само от тренировъчните данни**, за да няма leakage от теста.

- **Train:** примери, по които optimizer-ът обновява параметрите.
- **Validation:** отделни примери за избор на настройки, ако правим такава настройка.
- **Test:** примери, които остават недокоснати до финалната оценка.

Когато имаме множество изречения от един и същи документ, разделянето е най-добре да става **по документи**, а не произволно по изречения, за да не попаднат почти еднакви пасажии в train, и в test. При нашия малък набор резултатите ще са нестабилни; по-голям и разнообразен набор е необходим за убедителна оценка.

15. Ограничения и практически подобрения

Сегашно ограничение	Разумно надграждане
Без контекст между изреченията	Представяне на цял документ или контекстно-ориентиран модел в бъдещ проект.
Проста tokenization с пунктуация	По-добър tokenizer и последователна обработка на пунктуацията.
Непознати думи →	По-богат речник или subword tokenizer.
max_length = 10	Ясна политика за по-дълги изречения: подходящ лимит, truncation или друга архитектура.
top_k е фиксиран брой	Проверка на стойността; възможност за желан дял от оригиналния текст.
Няма сравнение на близки по смисъл изречения	Механизъм срещу повторения и redundant content.
Синтетични и малко на брой labels	По-разнообразни означени документи и отделен тестов набор.

Фокус на проекта. Не е необходимо всички подобрения да влязат в първата версия. TextBrief вече изпълнява учебната цел: проследим, работещ процес от входен текст до резюме със собствен обучен PyTorch модел.

16. Изпълнение и кратък чеклист

Командите се изпълняват от папката `AI/` в активирана виртуална среда, в която са инсталирани PyTorch и pandas.

```
PYTHONPATH=src python tests/test_model.py
PYTHONPATH=src python src/training/train.py
PYTHONPATH=src python src/inference/predict.py
PYTHONPATH=src python src/inference/summarize.py
```

- Тестът на модела проверява формите `[batch\_size]`, изчисляването на loss и наличието на градиенти след `backward()`.
- Обучението отпечатва среден loss за epoch и записва `artifacts/textbrief\_model.pt`.
- Inference възстановява същите параметри и речник; предсказването не обновява теглата.
- Summarizer избира `top\_k` изречения по оценка и ги връща в оригиналния им ред.

17. Ключови термини - за бърз преглед

Термин	Кратко значение
Token / tokenization	Елемент от текста / разделяне на текста на елементи.
Vocabulary / token ID	Речник, който свързва token с уникален целочислен индекс.
/	Липсваща token позиция / реален, но непознат token.
Embedding	Обучаем вектор за всеки token ID.
Masked mean pooling	Усреднява реалните token вектори до един вектор за изречение.
Logit / sigmoid	Сурова оценка / преобразуване в число между 0 и 1.
Criterion / loss	Функция за грешка / нейната стойност за batch-a.
Gradient / backward()	Информация за изменение на loss / изчисляването ѝ.
Optimizer / step()	Алгоритъм за обновяване / прилагане на обновяванията.
Batch / epoch	Група примери / едно преминаване през целия train набор.
Checkpoint / state_dict	Запазено състояние / речник с параметрите на модела.
Inference / eval()	Предсказване с научените параметри / режим за оценяване.

18. Къде се вписва Spring Boot?

Spring Boot е **следващата стъпка**, а не част от вече реализирания PyTorch модел. Примерна бъдеща архитектура: клиент → Spring Boot API → Python inference услуга → зареден TextBrief checkpoint → текстово резюме → отговор към клиента.

Най-простият практичен вариант е Python процес/HTTP услуга, който зарежда модела веднъж, а Spring Boot му изпраща текст и получава резултата. Не е необходимо `.pt`

checkpoint-ът да се зарежда директно в Java.

19. Заключение

TextBrief демонстрира завършен минимален AI pipeline: CSV → preprocessing → embeddings → sentence vectors → logits → loss и обучение → checkpoint → inference → extractive summary. Тестовите показват работеща интеграция на компонентите и няколко правилни избора на изречения; те не представляват измерване на надеждно качество за произволни документи.

Това е полезна основа за следващите курсове и проекти: математиката на градиентите, по-силни представяния на текст, контекстни модели и Computer Vision могат да бъдат надградени върху вече разбираня цикъл на обучение и inference.